

CS2.501 ADVANCED COMPUTER ARCHITECTURE · MONSOON 2026 · IIIT HYDERABAD

Mini-Project Part 1

Building a Big Matrix Multiply out of a 4×4×4 Unit

Parallel vs K-serial vs K-serial with double buffering: how fast, how big, which one to pick

Devansh Singh Tomar · Roll No. 2026203008 · Bluespec (bsc, Bluesim) · October 2026

1 The three designs at a glance

The unit computes $C += A \times B$ on 4×4 tiles (Int8 inputs, Int32 accumulator) behind a guarded request/response interface. One tile costs **9 requests** (4 LoadA, 4 LoadB, 1 Mul) and 16 ReadC requests drain each finished output tile. Every design obeys the §3 rule: its responses match a unit that handles one request at a time, in order.

	A · Combinational	B · K-serial	C · K-serial + double buffer
What Mul does	All 64 products and 16 four-term sums in one rule firing ($\ell = 1$)	One K-slice (16 MACs) per cycle: $K=0$ at the Mul request, $K=1..3$ in <code>r1_mul_step</code> ($\ell = 4$)	Same as B on the active bank; bank flag flips when $K=3$ finishes ($\ell = 4$)
Multipliers	64	16	16
A/B storage	1 bank	1 bank	2 banks + 1 flag (+256 bits)
While a Mul runs	Nothing to wait for	<code>rg_busy</code> blocks every request	Loads go to the inactive bank; Mul and ReadC wait
Rules	4, one per request type	5	12 (8 load rules, start, 2 step, ReadC)

2 Measurements

TABLE 1 · MEASURED (make, make depth)

Design	W1 cycles	W2 cycles	Stalls W1 / W2	Cells	logic_depth	Errors
A · combinational	40,961	299,009	1 / 1	113,160	71	0
B · K-serial	53,249	397,313	12,289 / 98,305	62,151	57	0
C · K-serial + DB	41,729	299,777	769 / 769	113,160 †	71 †	0

TABLE 2 · DERIVED (green = best in column)

Design	cycles × depth W1	cycles × depth W2	cells × depth	MACs/request (tile)	W1 run	W2 run
A	2,908,231	21,229,639	8,034,360	64/9 = 7.11	6.40	7.01
B	3,035,193 (+4.4%)	22,646,841 (+6.7%)	3,542,607	7.11	6.40	7.01
C †	2,962,759 (+1.9%)	21,284,167 (+0.3%)	8,034,360	7.11	6.40	7.01

Reading the tables. Cycles, stalls and cycles × depth rank **A < C < B**. Cells, depth and cells × depth rank **B < A** (C = A as recorded). The ranking flips between columns, and that is the trade-off: B is 30–33% slower in cycles but only 4–7% slower in estimated time, and 56% cheaper in cells × depth.

TABLE 3 · CELLS AND LOGIC DEPTH: PREDICTED RANKING VERSUS MEASURED

Quantity	Predicted order	Reason for the prediction	Measured	Verdict
Cells	$A > C \approx B$	A: 64 multipliers; B, C: 16; C adds a 2nd bank	$A \ 113,160 > B \ 62,151$;	$A > B$ right; B is 55% of A (not ¼).
Depth	$A > C \approx B$	A: multiply + 5-input sum; B: multiply + select + 1 add	$A \ 71 > B \ 57$;	$A > B$ right; B only 20% shallower (not ¼).

SIMULATION OUTPUT OF EACH DESIGN

Design A	<pre> workload requests cycles stalls errors ----- W1 40960 40961 1 0 W2 299008 299009 1 0 MINIPROJ OK (correct on both workloads) mini-project: PASS </pre>	40,961 / 299,009 cycles, 1 stall each: only the unavoidable end-of-run stall.
Design B	<pre> workload requests cycles stalls errors ----- W1 40960 53249 12289 0 W2 299008 397313 98305 0 MINIPROJ OK (correct on both workloads) </pre>	53,249 / 397,313 cycles; 12,289 / 98,305 stalls = 3 per Mul + 1
Design C	<pre> workload requests cycles stalls errors ----- W1 40960 41729 769 0 W2 299008 299777 769 0 MINIPROJ OK (correct on both workloads) </pre>	41,729 / 299,777 cycles; 769 stalls in both = 3 per output tile + 1.

3 Predictions versus measurement

Let ℓ be the cycles a Mul occupies the arithmetic. If the next tile's loads must wait, a tile takes $8 + \ell$ cycles. If they need not, it takes $\max(9, \ell) = 9$. At $\ell = 1$ both equal 9, so overlap buys nothing for A. At $\ell = 4$ they are 12 versus 9, a ratio of 4/3, which is the whole value of Design C. Predicted cycles = tiles $\times$ (cycles per tile) + 4,096 ReadC + 1 final stall.

Design	ℓ	cycles / tile	W1: predicted $\rightarrow$ measured	W2: predicted $\rightarrow$ measured	Verdict
A	1	$8 + \ell = 9$	40,961 $\rightarrow$ 40,961	299,009 $\rightarrow$ 299,009	exact
B	4	$8 + \ell = 12$	53,249 $\rightarrow$ 53,249	397,313 $\rightarrow$ 397,313	exact
C	4	$\max(9, \ell) = 9$	40,961 $\rightarrow$ 41,729	299,009 $\rightarrow$ 299,777	+768

- Design C – Cycle Count:** The initial formula missed the delay while ReadC waits for the final Mul to complete.
Each of the 256 output tiles adds 3 cycles, resulting in 768 extra cycles. The corrected formula is:
Cycles = (Number of Muls $\times$ 9) + 4,096 ReadC cycles + 768 extra cycles + 1
Therefore, **W1 takes 41,729 cycles** and **W2 takes 299,777 cycles**, matching the measured results.
- Cell Area and Depth:** The ranking $A > B$ was correct, but the improvement was smaller than expected.
Using four times fewer multipliers reduced cell area by only 45% and depth by 20%.

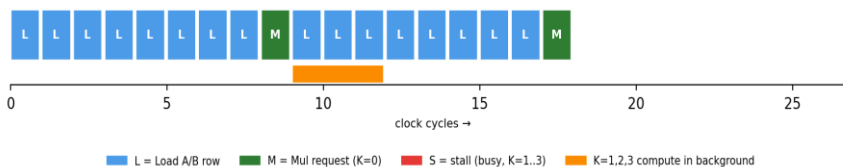
Design A - Mul finishes in 1 cycle (9 cycles per tile)



Design B - 3 stall cycles after every Mul (12 cycles per tile)



Design C - next tile loads into the other bank while K=1,2,3 compute (9 cycles per tile)



Two consecutive tiles: B loses 3 cycles after every Mul; C hides them behind the next tile's loads.

4 Answers to the questions

Q1	Did the prediction hold? Exact for A and B; off by +768 cycles for C. The cause is ReadC waiting on the last Mul of each output tile (Section 3).
Q2	Where do the cycles go? Stalls are 1 (A), 3 per Mul (B) and 3 per output tile (C), plus 1 at the end. A: 40,961 = 40,960 requests + 1. The one stall is unavoidable: the last response lands a cycle after the last request. B: 12,289 = 4,096 Muls × 3 + 1 (W2: 32,768 × 3 + 1 = 98,305). During K=1, 2, 3 the unit rejects every request, loads included. C: 769 = 256 output tiles × 3 + 1. The 3 leftover compute cycles hide behind the next tile's 8 loads (3 < 8); only ReadC waits.
Q3	Why is MACs per request the same for all three? What would change it? 7.11 for every design, because the interface fixes it, not the hardware. A 32-bit request carries four Int8 values, so a 4×4 tile always needs 8 loads and 1 Mul for 64 MACs. (Whole runs give 6.40 for W1 and 7.01 for W2, because ReadC requests do no MACs.) A, B and C change only how many cycles each request takes, never how many requests exist. To move it: carry more data per request; use a bigger tile (S×S×S via registers gives $S^3/(S^2/2+1)$, e.g. 15.5 at S = 8); or stop sending data through registers (LoadTile with address and stride makes a tile 3 requests: $64/3 \approx 21$).
Q4	What sets the longest path? The multiplier. Multiplier count sets area, not depth. Along one output the chain is sign-extend → 8×8 multiply → sum of four products plus old C. B still has a full multiplier, a 4:1 select and one accumulate add, and already reaches depth 57, so summing over K adds only 71 – 57 = 14 levels to A. The trap: B has 4× fewer multipliers but only a 20% shorter path, because multipliers work in parallel. Cells fall just 45% since registers, FIFOs and adders do not shrink. A truly shorter path needs the multiply itself pipelined.
Q5	What happens when a LoadA arrives during a Mul? Either stall it (B) or write the other bank (C); Mul and ReadC always wait. Writing A mid-Mul would change A under the running Mul and break §3. Stall (B): guard every rule with !rg_busy. Costs 8–1 = 3 cycles per Mul (12,288 on W1), no extra hardware. Double buffer (C): write the inactive bank and flip the flag only when K=3 retires (K=1..3 still read the active bank). Costs +256 bits and 2:1 operand muxes. A snapshot copy at Mul start costs about the same. A Mul (writes C) and a ReadC (needs the finished C) must wait in both designs.
Q6	Why does W2 cost more than W1 per unit of readout, but less per MAC? Readout is amortised over 8× more work in W2. Both read out the same 4,096 elements, but W2 does 128 Muls per output tile against 16. For A: 10.0 cycles per readout in W1 (40,961 / 4,096) versus 73.0 in W2 (299,009 / 4,096). Readout does no MACs (10% of W1 requests, 1.4% of W2), so cycles per MAC fall: A 0.156 → 0.143, B 0.203 → 0.189, C 0.159 → 0.143. C is the most sensitive (–10.2% vs –8.7% for A, –6.7% for B): its only loss is a per-output-tile cost (768 cycles = 1.9% of W1, 0.26% of W2) that deeper accumulation amortises. B's loss is per Mul, so it stays near 30–33%.
Q7	How wide must the accumulator be? $W_ACC \geq 2 \cdot W_ELEM + \lceil \log_2(K \cdot M) \rceil$. At 16 bits, zero Muls are safe. One product is at most 2^{2W_ELEM-2} ($-128 \times -128 = 2^{14}$). Summing $K \cdot M$ of them must stay $\leq 2^{W_ACC-1}-1$, so $M \leq \lfloor (2^{W_ACC-1}-1) / (K \cdot 2^{2W_ELEM-2}) \rfloor$ (the width formula is safe, at most one bit conservative). At $W_ACC = 32$: 32,767 Muls (W1 and W2 use 16 and 128). At $W_ACC = 16$: 0, because one Mul can add $4 \times 16,384 = 65,536$ (one Mul needs 18 bits). This is why a row of C takes four 32-bit reads, and why readout, which also zeroes C, exists.
Q8	When does a DMA-fed unit stop being instruction-bound? Immediately: a 4×4×4 unit needs about 32 GB/s. DMA relieves instruction supply (one descriptor names a tile), but a 4×4×4 tile moves 32 bytes for 64 MACs (2 MACs/byte; S/2 for an S-tile). At 64 MACs/cycle and 1 GHz that is 32 GB/s, more than one DDR4-3200 channel (about 25.6 GB/s, my example figure). Needed bandwidth is 128/S bytes/cycle, so $S \geq 5$ (S = 8 in practice, 16 GB/s) fits within one channel. OS guarantee §2 never needed: page pinning (pages must not move or be swapped during a transfer). Cache coherence and address translation are the others.

Q9

When BSV does not do what you expect (make schedule)**Mutually exclusive guards and disjoint registers let rules fire together.**

bsc schedules whole rules, so each rule handles one request type. In B, `r1_mul_start` and `r1_mul_step` both write `rg_c`, `rg_busy`, `rg_k`, but their guards (`!rg_busy`, `rg_busy`) are exclusive, which -aggressive-conditions lets bsc see. In C, the bank-1 load rules and `r1_mul_bank0` touch different registers, so they fire in the same cycle; request rules exclude each other because they all dequeue `f_req`.

5 My choice, and the strongest argument against it

Chosen: Design B (K-serial). Priority: lowest hardware cost per unit of time (`cells × logic_depth`). The unit is meant to sit beside a CPU as a custom instruction, where area and power are limited.

Why B

- **Smallest and shallowest:** -45% cells, -20% depth.
- **Best cells × depth:** 3.54M vs 8.03M (-56%).
- **Time penalty is small:** within 4.4% (W1) and 6.7% (W2) of the fastest.
- **Simplest:** 2 extra registers and 5 rules, so easiest to keep correct under §3.

Strongest argument against: Design C

- Only 1.9% (W1) and 0.26% (W2) behind A in cycles, and 25% fewer cycles than B on W2 (299,777 vs 397,313), for just 256 more bits.
- **My defence:** C's edge needs a request every cycle. In Part 2 a CPU must fetch, decode and execute each request, so B's 3 blocked cycles are mostly hidden. Under my priority C cannot beat B anyway (same multipliers plus extra storage).
- **I concede:** for sustained back-to-back streaming, C is better.

WHEN TO USE EACH DESIGN

Design	Best when...	Main cost
A	Latency per Mul or simplicity matters and area and power are free; also as the golden reference.	64 multipliers and the longest path.
B	Area, power or cost is limited, or requests arrive at least 4 cycles apart.	About 30% more cycles under continuous traffic; interface idle during $K=1..3$; ReadC and loads blocked while busy.
C	Sustained tile streaming (like W1 and W2) where loading a tile takes at least as long as computing it ($8 \geq 4$ here).	Extra storage and more complex control; ReadC still waits.